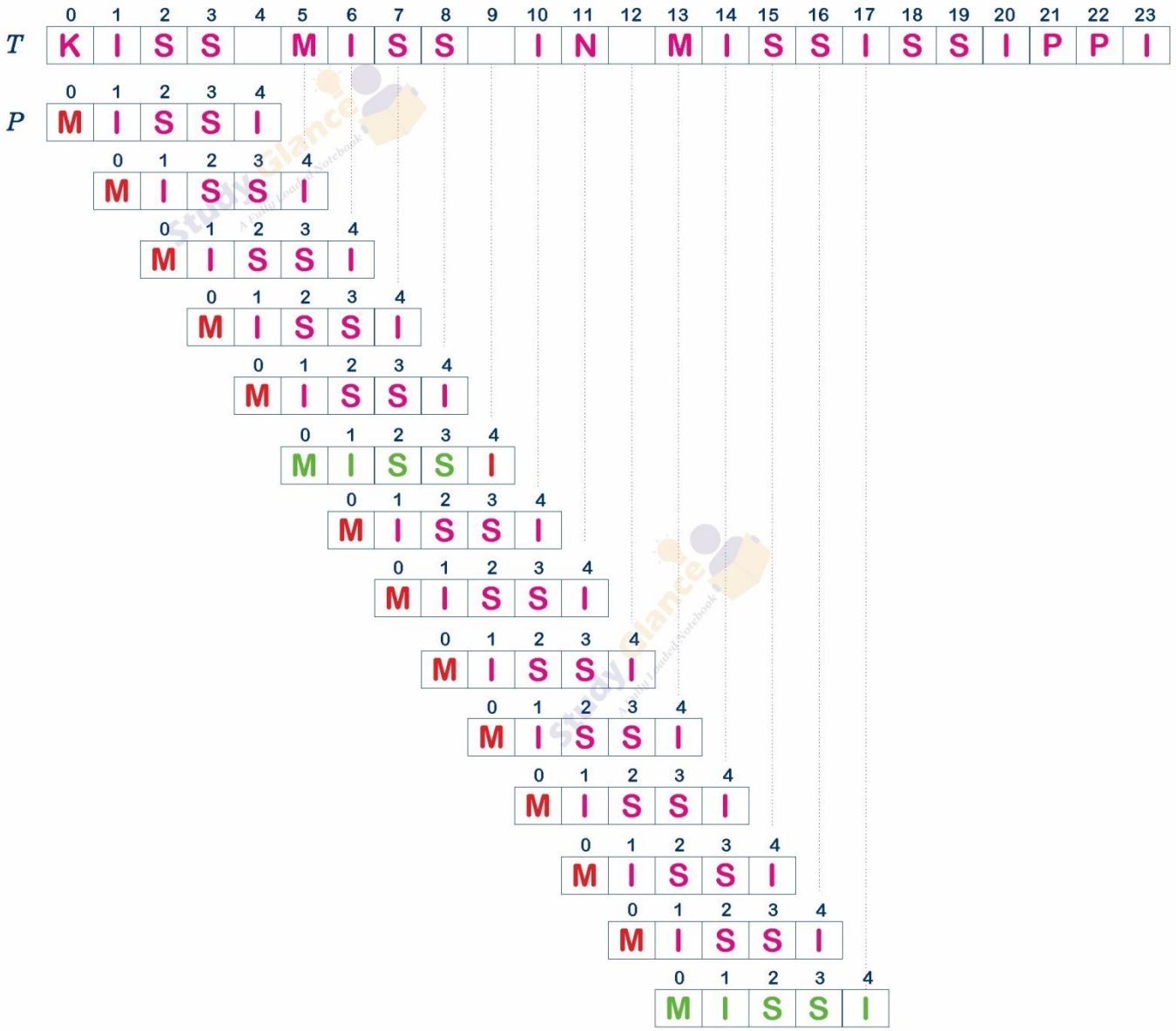
Brute force Pattern Matching

A brute force algorithm is a straight forward approach to solving a problem.

- It is based on **trial and error** where the programmer tries to merely utilize the computer's fast processing power to solve a problem, rather than applying some advanced algorithms and techniques developed with human intelligence.
- It might increase both space and time complexity.
- A simple example of applying brute force would be linearly searching for an element in an array. When each and every element of an array is compared with the data to be searched, it might be termed as a brute force approach, as it is the most direct and simple way one could think of searching the given data in the array

Brute Force Pattern Matching Algorithm

1. Start at the beginning of the text and slide the pattern window over it.
2. At each position of the text, compare the characters in the pattern with the characters in the text.
3. If a mismatch is found, move the pattern window one position to the right in the text.
4. Repeat steps 2 and 3 until the pattern window reaches the end of the text.
5. If a match is found (all characters in the pattern match the corresponding characters in the text), record the starting position of the match.
6. Move the pattern window one position to the right in the text and repeat steps 2-5.
7. Continue this process until the pattern window reaches the end of the text.


T
K I S S M I S S I N M I S S I S S I P P I
P
M I S S I

Boyer–Moore Pattern Matching

The **Boyer–Moore Pattern Matching algorithm** is one of the most efficient string-searching algorithm that is the standard benchmark for practical pattern matching. It was developed by **Robert Stephen Boyer** and **J Strother Moore** in the year **1977**.

The Boyer-Moore algorithm works by **pre-processing** the pattern and then scanning the text from right to left, starting with the rightmost characters. It is based on the principle that if a mismatch is found, there is no need to match the remaining characters. This backwards approach significantly reduces the algorithm's time complexity compared to naive string search methods.

Here's a step-by-step explanation of how the Boyer-Moore algorithm works:

1. Pre processing

Boyer-Moore algorithm uses two heuristics for pre-processing the pattern:

a. Bad Character Heuristic
b. Good Suffix Heuristic

Bad Character Heuristic (Building Bad Character Table)

Determines how far the pattern can be shifted when a character mismatch occurs.

a. Create an array (often called the "**Bad Character Table**") to store the rightmost occurrence of each character in the pattern. If a character is not in the pattern, its value is set to **pattern length**.
b. For each character in the pattern, update the table with its last occurrence in the pattern with
pattern length - index - 1.
BCT(ch)={m if ch is not in p
m-j-1otherwise

Example:

j →	0	1	2	3	4
P	M	I	S	S	I

Length(m)=5

Bad Character Table

M	I	S	*
4	0	1	5

M → (m-j-1) = 5-0-1 = 4
I → (m-j-1) = 5-1-1 = 3
S → (m-j-1) = 5-2-1 = 2
S → (m-j-1) = 5-3-1 = 1
I → (m-j-1) = 5-4-1 = 0

2. Searching

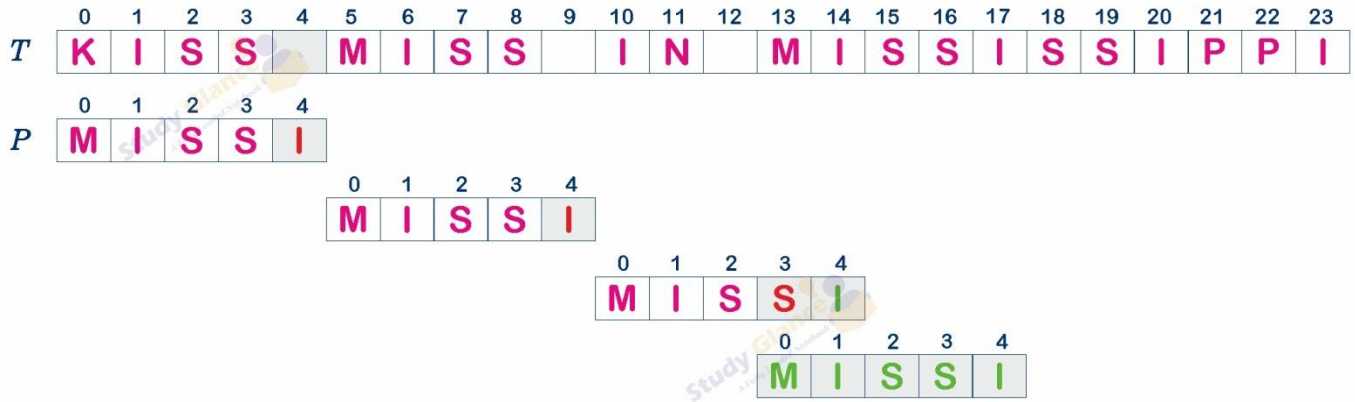
Once the pre processing is done, the actual search begins:

a. Align the pattern with the beginning of the text.
b. Compare the pattern with the text from right to left.
c. If all characters of the pattern match, a valid occurrence is found.

3. Shifting the Pattern:

When a mismatch occurs

- **If the character is not in the pattern:** Shift the pattern such that the mismatched character in the text aligns with the rightmost occurrence of it in the pattern.
- **If the character is not in the pattern:** shift the entire pattern past the character.



3. Repeat Comparison

After shifting the pattern according to the above rules, repeat the comparison process:

a. Continue comparing the pattern with the text from right to left.
b. Apply the shifting rules whenever a mismatch is encountered.
c. Continue this process until the end of the text is reached or all occurrences of the pattern are found.
d.

4. Termination

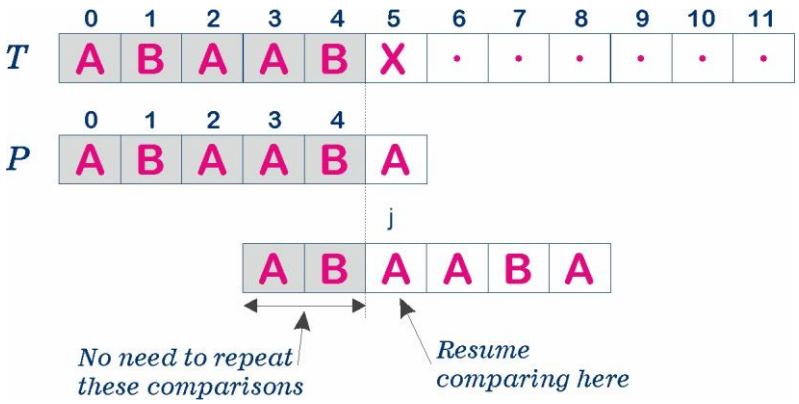
The algorithm terminates when either

- Th3E e pattern has been shifted past the **end of the text**, indicating no more matches are possible.
- All occurrences of the **pattern has have been found**.

KMP Pattern Matching

The **Knuth-Morris-Pratt (KMP)** pattern matching algorithm is an efficient string searching method developed by **Donald Knuth**, **James H. Morris**, and **Vaughan Pratt** in the year 1970. It is used to find the occurrences of a "pattern" within a "text" without checking every single character in the text, which is a significant improvement over the brute-force approach.

The KMP algorithm compares the pattern to the text in left-to-right, but shifts the pattern, P more intelligently than the brute-force algorithm. When a mismatch occurs, what is the most we can shift the pattern so as to avoid redundant comparisons. The answer is that the largest prefix of P[0..j] that is a suffix of P[1.j].

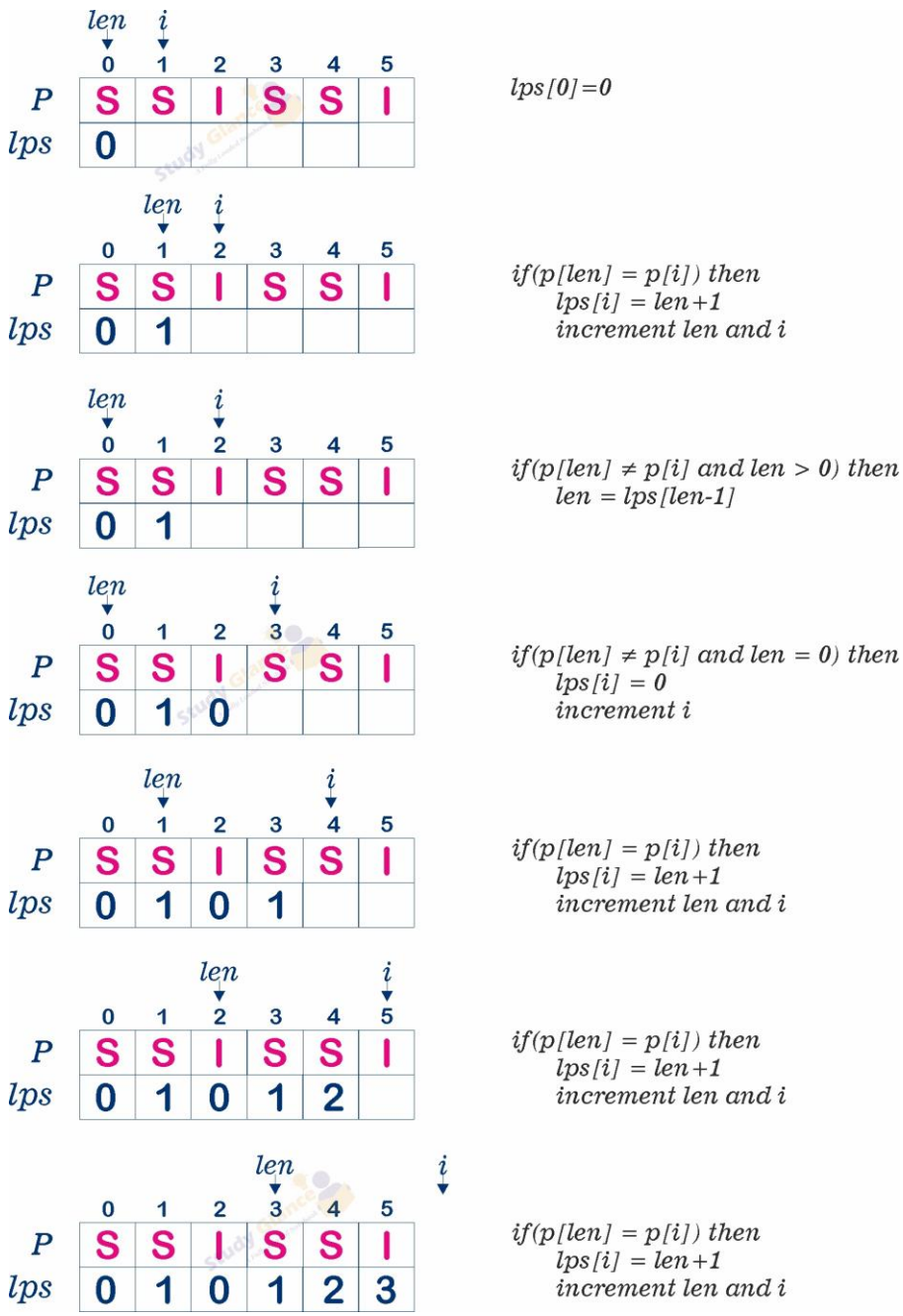


Here's a step-by-step explanation of how the KMP algorithm works:

1. Preprocessing(Building the LPS Array)

The core idea to preprocess the pattern to construct an **LPS(Longest Prefix Suffix)** array. This array stores the length of the longest proper prefix which is also a suffix for each sub-pattern of the pattern. This preprocessing helps in determining the next positions in the pattern to be compared, thus avoiding redundant comparisons.

1. Start by initializing the first element of **lps[]** to 0, as a single character can't have any proper prefix or suffix.
2. Maintain two pointers, **len** and **i**, where **len** is the length of the last longest prefix suffix. Initially, **len := 0** and **i := 1**.
3. Repeat steps 4 to 6, **while (i < m)**
4. If **pattern[len]** equals **pattern[i]**, set **lps[i] = len + 1**, increment both **i** and **len**.
5. If they don't match and len is not 0, update len to **lps[len - 1]**.
6. If they don't match and len is 0, set **lps[i] = 0** and increment i.



2. Searching

Once the preprocessing is done, the actual search begins:

a. Align the pattern with the beginning of the text.

b. Compare the pattern with the text from left to right.

c. If all characters of the pattern match, a valid occurrence is found.

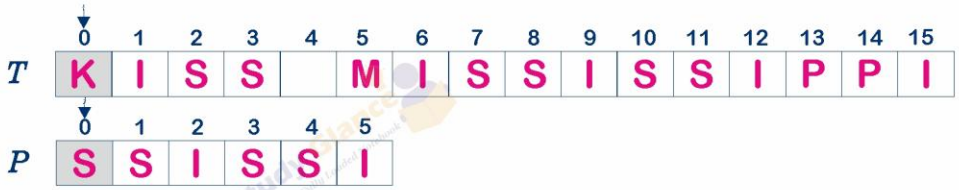
3. Shifting the Pattern:

- Compare pattern[j] with text[i].
- If they match, increment both i and j.

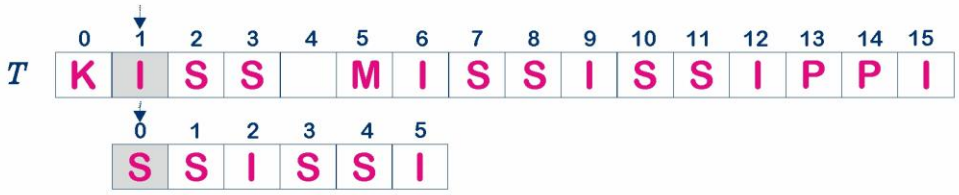
- If j equals the pattern length, a match is found. Optionally report the match, then set j to lps[j - 1].
- If they don't match and j is not 0, set j to lps[j - 1]. Do not increment i here.
- If they don't match and j is 0, increment i

4. Repeat Comparison

a. Continue comparing the pattern with the text from left to right.
b. Apply the shifting rules whenever a mismatch is encountered.
c. Continue this process until the end of the text is reached or all occurrences of the pattern are found.



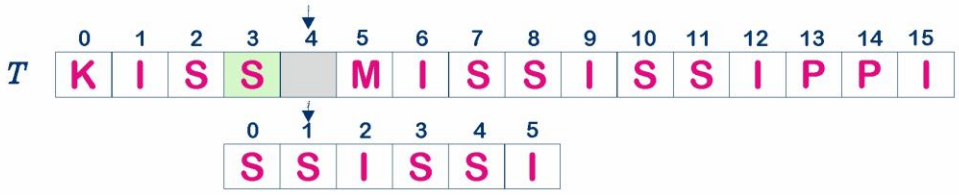
if(P(j) ≠ T(i) and j = 0) then increment i



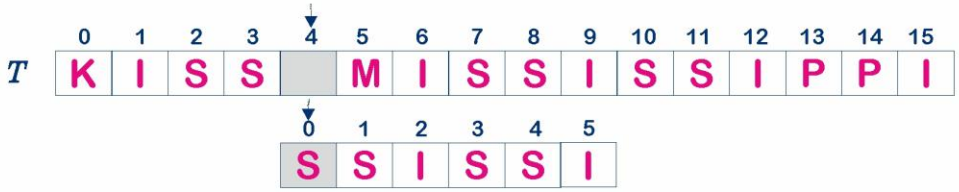
if(P(j) ≠ T(i) and j = 0) then increment i

if(P(j) = T(i) and j ≠ m-1) then increment i and j

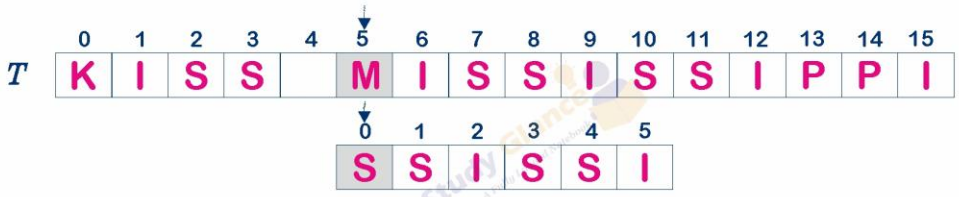
if(P(j) ≠ T(i) and j > 0) then j = lps[j-1]



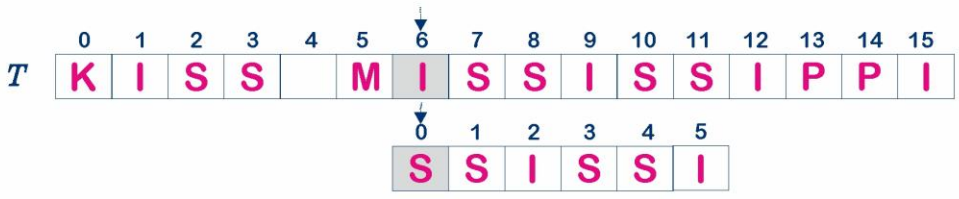
if(P(j) ≠ T(i) and j > 0) then j = lps[j-1]



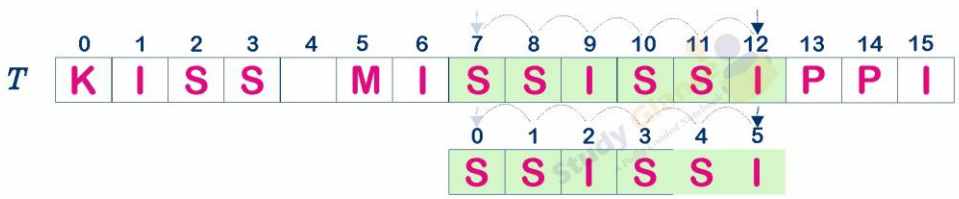
if(P(j) ≠ T(i) and j = 0) then increment i



if(P(j) ≠ T(i) and j = 0) then increment i



if(P(j) ≠ T(i) and j = 0) then increment i



if(P(j) = T(i) and j = m-1) then Pattern found

5. Termination

The algorithm terminates when either

- The pattern has been shifted past the **end of the text**, indicating no more matches are possible.
- All occurrences of the **pattern has have been found**.

TRIES DATA STRUCTURE

Trie is an efficient information re***Trie***val data structure. The term tries comes from the word retrieval

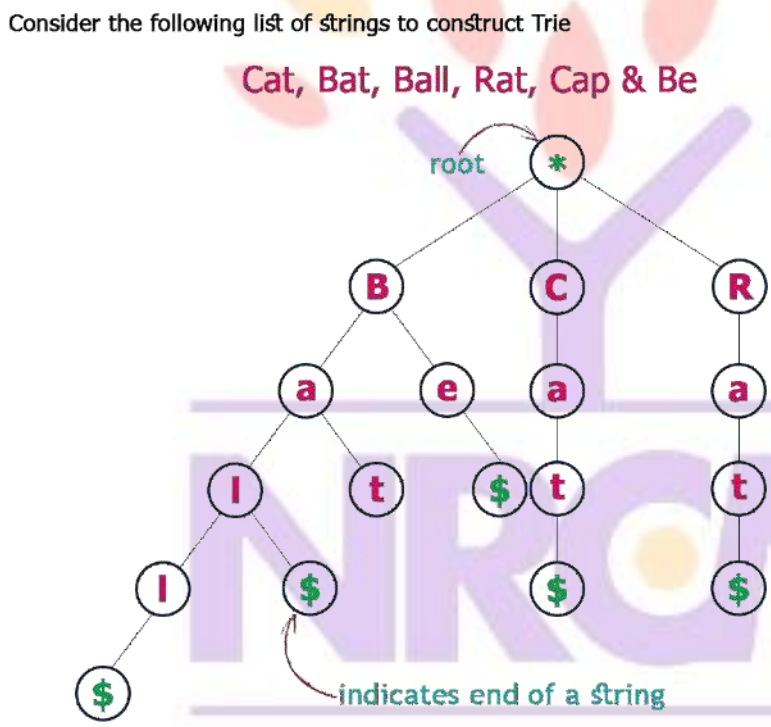
Definition of a Trie

- Data structure for representing a collection of strings
- In computer science , a trie also called digital tree or radix tree or prefix tree.
- Tries support fast string matching.

Properties of Tries

- A Multi way tree
- Each node has from 1 to n children
- Each edge of the tree is labeled with a character
- Each leaf node corresponds to the stored string which is a concatenation of characters on a path from the root to this node.

EXAMPLE



Trie | (Insert and Search)

Trie is an efficient information retrieval data structure. Using Trie, search complexities can be brought to an optimal limit (key length).
Given multiple strings. The task is to insert the string in a Trie

Examples:

Example 1: str = {"cat", "there", "caller", "their", "calling", “bat”}

```
  root
 /    \
c      t
| |     a
  h
| \ |
l t e | | \ l
i r
| \       | |
e i r e
| |
r n
  |
  g
```

Example 2: str = {"Candy", "cat", "Caller", "calling"}

```
  root
```

your roots to success...

```
    |
    c
    |
    a
  / | \
  l   n t
  | |   l
    d
  | \ |
  e i y
  | |
  r n
    |
    g
```

Approach: An efficient approach is to treat every character of the input key as an individual trie node and insert it into the trie. Note that the children are an array of pointers (or references) to next level trie nodes. The key character acts as an index into the array of children. If the input key is new or an extension of the existing key, we need to construct non-existing nodes of the key, and mark end of the word for the last node. If the input key is a prefix of the existing key in Trie,

we simply mark the last node of the key as the end of a word. The key length determines Trie depth.

Trie deletion

Here is an algorithm how to delete a node from trie.

During delete operation we delete the key in bottom up manner using recursion. The following are possible conditions when deleting key from trie,

1. Key may not be there in trie. Delete operation should not modify trie.
2. Key present as unique key (no part of key contains another key (prefix), nor the key itself is prefix of another key in trie). Delete all the nodes.
3. Key is prefix key of another long key in trie. Unmark the leaf node.
4. Key present in trie, having atleast one other key as prefix key. Delete nodes from end of key until first leaf node of longest prefix key.

Time Complexity: The time complexity of the deletion operation is O(n) where n is the key length

Advantages of Trie Data Structure

Tries is a tree that stores strings. The maximum number of children of a node is equal to the size of the alphabet. Trie supports search, insert and delete operations in O(L) time where **L** is the length of the key.

Hashing:- In hashing, we convert the key to a small value and the value is used to index data. Hashing supports search, insert and delete operations in O(L) time on average.

Self Balancing BST : The time complexity of the search, insert and delete operations in a selfbalancing Binary Search Tree (BST) (like Red-Black Tree, AVL Tree, Splay Tree, etc) is O(L * Log n) where n is total number words and L is the length of the word. The advantage of Selfbalancing BSTs is that they maintain order which makes operations like minimum, maximum, closest (floor or ceiling) and kth largest faster.

Why Trie? :-

1. With Trie, we can insert and find strings in ***O(L)*** time where *L* represent the length of a single word. This is obviously faster than BST. This is also faster than Hashing because of the ways it is implemented. We do not need to compute any hash function. No collision handling is required (like we do in open addressing and separate chaining)

2. Another advantage of Trie is, we can easily print all words in alphabetical order which is not easily possible with hashing.

3. We can efficiently do prefix search (or auto-complete) with Trie.

Issues with Trie :-

The main disadvantage of tries is that they need a lot of memory for storing the strings. For each node we have too many node pointers(equal to number of characters of the alphabet), if space is concerned, then **Ternary Search Tree** can be preferred for dictionary implementations. In Ternary Search Tree, the time complexity of search operation is O(h) where h is the height of the tree. Ternary Search Trees also supports other operations supported by Trie like prefix search, alphabetical order printing, and nearest neighbor search.

The final conclusion is regarding *tries data structure* is that they are faster but require *huge memory* for storing the strings.

APPLICATIONS OF TRIES

String handling and processing are one of the most important topics for programmers. Many real time applications are based on the string processing like:

1. Search Engine results optimization 2. Data Analytics 3. Sentimental Analysis

The data structure that is very important for string handling is the **Trie** data structure that is based on **prefix of string**

TYPES OF TRIES

Tries are classified into three categories:

1. Standard Tries
2. Compressed Tries
3. Suffix Tries

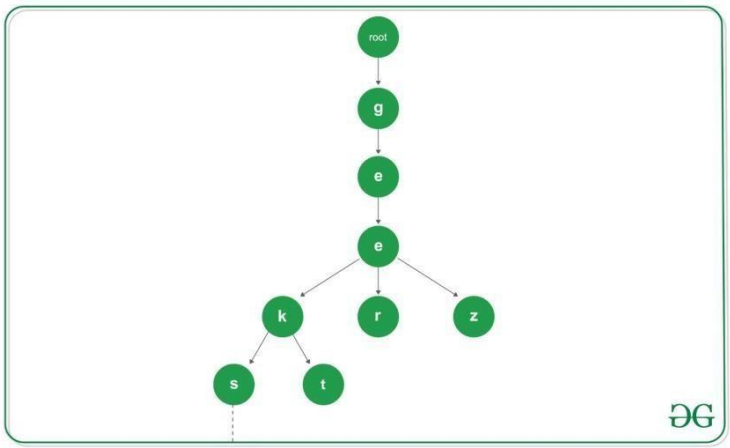
STANDARD TRIES

A standard trie have the following properties:}

- It is an ordered tree like data structure.
- Each node(except the root node) in a standard trie is labeled with a character.
- The children of a node are in alphabetical order.
- Each node or branch represents a possible character of keys or words.
- Each node or branch may have multiple branches.

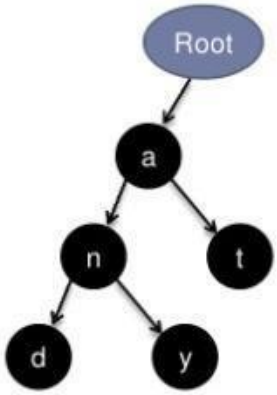
- The last node of every key or word is used to mark the end of word or node.
- The path from external node to the root yields the string of S.

Below is the illustration of the Standard Trie



Standard Trie Insertion

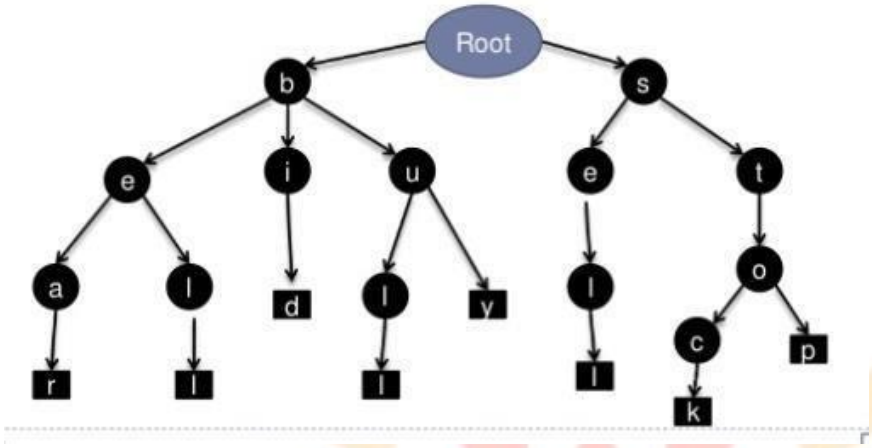
Strings={ a,an,and,any}



Example of Standard Trie

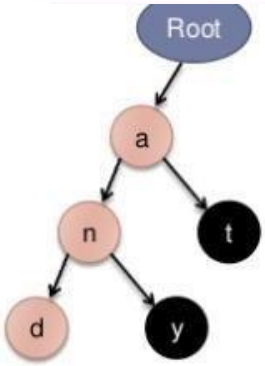
Standard trie for the following strings

S={ bear, bell, bid, bull, buy, sell, stock, stop}



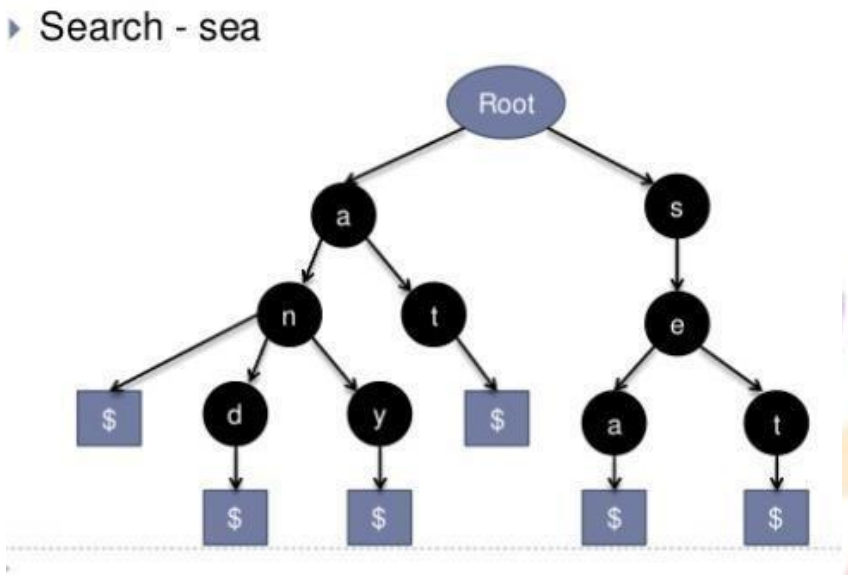
Handling Keys(strings)

- When a key is prefix of another key
  How can we know that “an “ is a word
  Example : an, and



Standard Trie Searching

Search hit where search node has a $ symbol



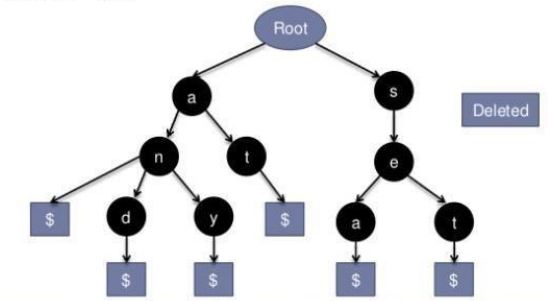
Standard Trie Deletion

To perform the deletion there exist cases

1. Word not found
   Return false
2. Word exist as a standalone word
   I. Part of any other node
   Example:

your roots to success...

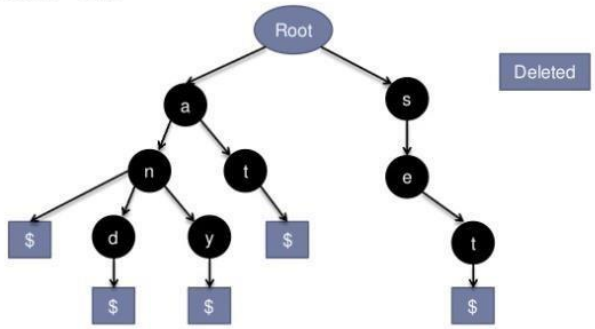
Delete - sea



II. Does not part of any other node

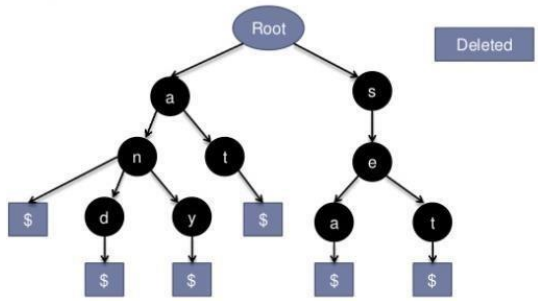
EXAMPLE

Delete - set



3. Word exist as a prefix of another word.

Delete - an



COMPRESSED TRIE

A Compressed trie have the following properties:

1. A Compressed Trie is an advanced version of the standard trie.

2. Each nodes(except the **leaf** nodes) have atleast 2 children.

3. It is used to achieve space optimization.

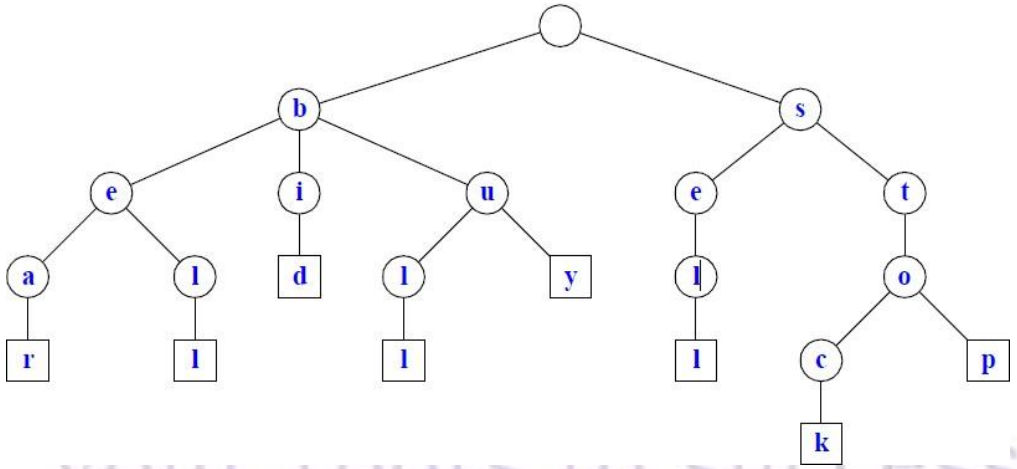
4. To derive a Compressed Trie from a Standard Trie, compression of chains of redundant nodes is performed.
5. It consists of grouping, re-grouping and un-grouping of keys of characters.

6. While performing the insertion operation, it may be required to un-group the already grouped characters.

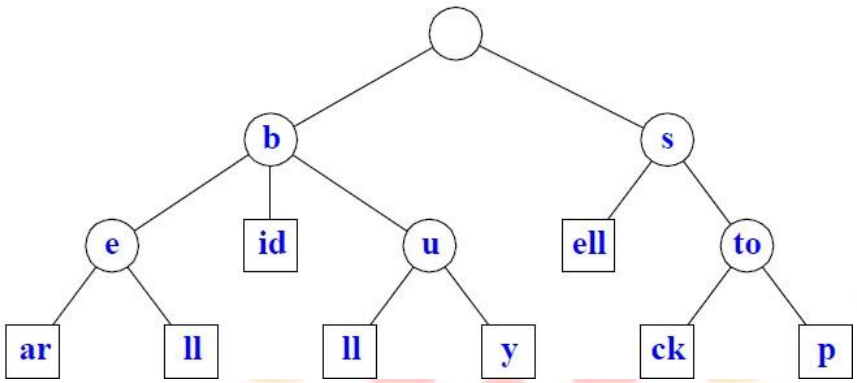
7. While performing the deletion operation, it may be required to re-group the already grouped characters.

Compressed trie is constructed from standard trie

- Standard Trie:



- Compressed Trie:



Storage of Compressed Trie

A compressed Trie can be stored at O9s) where s= | S| by using O(1) Space index ranges at the nodes

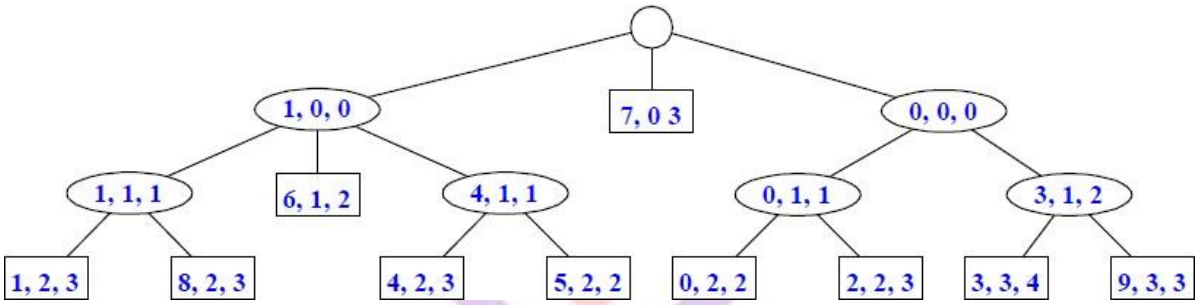
In the below representation each node is represented with (I,j,k) value
I -----indicate index of the string j—starting index of the character of
string I k -- ending index of the character of the string I
Ex: In the given diagram node (4,2,3) having the characters**(ll)** which belongs to s[4] so i=4, index of l character in s[4] is 2 so j=2 and ending index is 3 so k=3

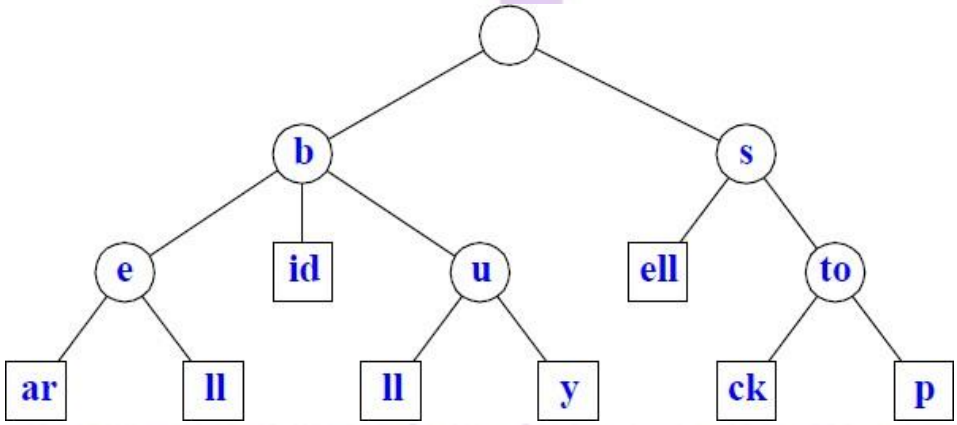
your roots to success...

	0	1	2	3	4
$S[0]$ =	s	e	e		
$S[1]$ =	b	e	a	r	
$S[2]$ =	s	e	l	l	
$S[3]$ =	s	t	o	c	k

	0	1	2	3
$S[4]$ =	b	u	l	l
$S[5]$ =	b	u	y	
$S[6]$ =	b	i	d	

	0	1	2	3
$S[7]$ =	h	e	a	r
$S[8]$ =	b	e	l	l
$S[9]$ =	s	t	o	p





your roots to success...

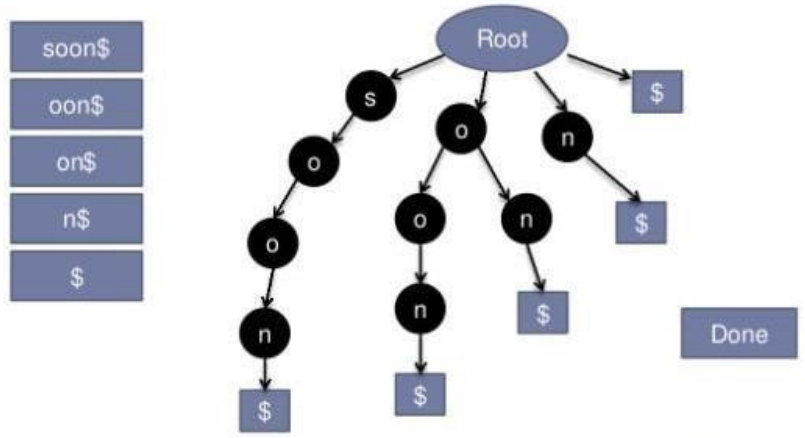
SUFFIX TRIES

A Suffix trie have the following properties:

1. Suffix trie is a compressed trie for all the suffixes of the text
2. Suffix trie are space efficient data structure to store a string that allows many kinds of queries to be answered quickly.

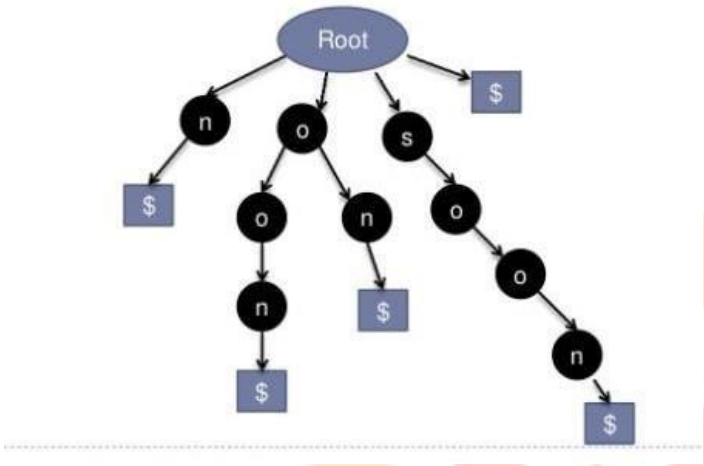
Example

Let us consider an example text “soon$”



After alphabetically order the trie look like

your roots to success...



Advantages of suffix tries

1. Insertion is faster compared to the hash table
2. Look up is faster than hash table implementation
3. There are no collision of different keys in tries

your roots to success...